# Labo3 Geavanceerde Computer Architecture

Arne Van der Stede en Syme Vandenbosch

Academiejaar 2025-2026
March 4, 2026

## 1 Introduction

In dit labo onderzoeken we twee prestatie aspecten bij het programmeren van GPU's.

- de invloed van het aantal actieve threads en warps op de uitvoeringstijd
- de inpact van thread divergence binnen warps

In het eerste gedeelte wordt een simpele omkerings (inversie) kernel ontwikkeld die een RGB afbeelding omzet naar het inverse. Door de kleurcomponent $x$ te vervangen door $255 - x$. De rekentijd wordt bekeken voor verschillende aantal van threads (Dus bij het aantal volledig/gedeeltelijk gevulde warps). Door het toepassen van striding kan de kernel efficiënter werken met minder threads dan dat er pixels zijn.

In het tweede deel wordt de omkeringskernel uitgebreidt met een 1D gaussische vervaging die toegepast wordt op enkel de rode component in de afbeelding. Groen en blauw worden gewoon nogsteeds geïnverteerd. Dan worden layout geherstructureerd van RGBRGB $->$ RRRRRGGGGGBBBBB. En dan wordt er gekeken wat de tijdswinst is met divergence t.o.v. zonder divergence.

## 2 Code

### 2.1 Warps

```
/*
 * Code snippet for importing / exporting image data.
 *
 * To convert an image to a pixel map, run 'convert <name>.<extension> <name>.ppm
 *
 */
#include <cstdint>      // Data types
#include <iostream>     // File operations

// #define M 512        // Lenna width
// #define N 512        // Lenna height
#define M 960        // VR width
#define N 1280        // VR height
#define C 3          // Colors
#define OFFSET 16    // Header length


uint8_t* get_image_array(void){
    /*
     * Get the data of an (RGB) image as a 1D array.
     *
     * Returns: Flattened image array.
     *
     * Noets:
     *  - Images data is flattened per color, column, row.
     *  - The first 3 data elements are the RGB components
     *  - The first 3*M data elements represent the firts row of the image
     *  - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M}, r_{1,0}, ..., b_{b,M},
         ..., b_{N,M}
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile=fopen("./anna.ppm","rb");
    if(imageFile==NULL){
```

```
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Initialize empty image array
    uint8_t* image_array = (uint8_t*)malloc(M*N*C*sizeof(uint8_t)+OFFSET);

    // Read the image
    fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t)+OFFSET, imageFile);

    // Close the file
    fclose(imageFile);

    // Move the starting pointer and return the flattened image array
    return image_array + OFFSET;
}


void save_image_array(uint8_t* image_array){
    /*
     * Save the data of an (RGB) image as a pixel map.
     *
     * Parameters:
     *  - param1: The data of an (RGB) image as a 1D array
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile=fopen("./output_image.ppm","wb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Configure the file
    fprintf(imageFile,"P6\n");               // P6 filetype
    fprintf(imageFile,"%d %d\n", M, N);      // dimensions
    fprintf(imageFile,"255\n");              // Max pixel

    // Write the image
    fwrite(image_array, 1, M*N*C, imageFile);

    // Close the file
    fclose(imageFile);
}


__global__ void invert_kernel(uint8_t* image, int size) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;

    for (int i = index; i < size; i += stride) {
        image[i] = 255 - image[i];
    }
}

int main (void) {

    // Read the image
    uint8_t* image_array = get_image_array();

    // Allocate output
    uint8_t* new_image_array = (uint8_t*)malloc(M*N*C);

    // CUDA memory and setup
    int size = M * N * C;
    uint8_t* d_image;
    cudaMalloc(&d_image, size);

    // Warm-up and event creation
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    invert_kernel<<<1, 1>>>(d_image, size);
    cudaDeviceSynchronize();

    // Parameters
    int thread_counts[] = {1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024};
```

```
    int num_tests = sizeof(thread_counts) / sizeof(int);
    int iterations = 100;

    std::cout << "TPB\tTotal Threads\tAvg Time (ms)" << std::endl;

    for (int t = 0; t < num_tests; t++) {
        int threads_per_block = thread_counts[t];
        int total_kernel_threads = size / 16;
        int blocks_per_grid = (total_kernel_threads + threads_per_block - 1) /
            threads_per_block;

        float total_time = 0;
        for (int i = 0; i < iterations; i++) {

            cudaMemcpy(d_image, image_array, size, cudaMemcpyHostToDevice);

            cudaEventRecord(start);
            invert_kernel<<<blocks_per_grid, threads_per_block>>>(d_image, size);
            cudaEventRecord(stop);

            cudaEventSynchronize(stop);
            float ms = 0;
            cudaEventElapsedTime(&ms, start, stop);
            total_time += ms;
        }

        std::cout << threads_per_block << "\t" << threads_per_block * blocks_per_grid <<
            "\t\t" << total_time / iterations << std::endl;
    }

    // Save the image
    cudaMemcpy(new_image_array, d_image, size, cudaMemcpyDeviceToHost);
    save_image_array(new_image_array);

    // Cleanup
    cudaFree(d_image);
    cudaEventDestroy(start);
    cudaEventDestroy(stop);
    free(new_image_array);

    return 0;
}
```

Listing 1: Code voor de warps

## 2.2 Thread divergence

### planar vs interleaved

```
/*
 * Code snippet for importing / exporting image data.
 *
 * To convert an image to a pixel map, run 'convert <name>.<extension> <name>.ppm
 *
 */
#include <cstdint>      // Data types
#include <iostream>     // File operations

// #define M 512        // Lenna width
// #define N 512        // Lenna height
#define M 960        // VR width
#define N 1280        // VR height
#define C 3          // Colors
#define OFFSET 16    // Header length


uint8_t* get_image_array(void){
    /*
     * Get the data of an (RGB) image as a 1D array.
     *
     * Returns: Flattened image array.
     *
     * Noets:
     *  - Images data is flattened per color, column, row.
     *  - The first 3 data elements are the RGB components
     *  - The first 3*M data elements represent the firts row of the image
     *  - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M}, r_{1,0}, ..., b_{b,M},
          ..., b_{N,M}
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile=fopen("./anna.ppm","rb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Initialize empty image array
    uint8_t* image_array = (uint8_t*)malloc(M*N*C*sizeof(uint8_t)+OFFSET);

    // Read the image
    fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t)+OFFSET, imageFile);

    // Close the file
    fclose(imageFile);

    // Move the starting pointer and return the flattened image array
    return image_array + OFFSET;
}


void save_image_array(uint8_t* image_array, const char* filename){
    /*
     * Save the data of an (RGB) image as a pixel map.
     *
     * Parameters:
     *  - param1: The data of an (RGB) image as a 1D array
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile=fopen(filename,"wb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Configure the file
    fprintf(imageFile,"P6\n");               // P6 filetype
    fprintf(imageFile,"%d %d\n", M, N);      // dimensions
    fprintf(imageFile,"255\n");              // Max pixel

```

```
    // Write the image
    fwrite(image_array, 1, M*N*C, imageFile);

    // Close the file
    fclose(imageFile);
}
// --- Version 1: Interleaved (Non-Planar) Kernels ---
// Data layout: [R0, G0, B0, R1, G1, B1, ...]
__global__ void invert_interleaved(uint8_t* data, int num_pixels) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < num_pixels) {
        // Strided access: i*3
        data[i * 3 + 1] = 255 - data[i * 3 + 1]; // Invert Green
        data[i * 3 + 2] = 255 - data[i * 3 + 2]; // Invert Blue
    }
}

__global__ void gaussian_interleaved(uint8_t* input, uint8_t* output, int num_pixels) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < num_pixels) {
        int r = i * 3;
        float val = 0.50f * input[r];

        if (i >= 1) {
            val += 0.25f * input[r - 3];
        } else {
            val += 0.25f * input[r];
        }

        if (i >= 2) {
            val += 0.10f * input[r - 6];
        } else {
            val += 0.10f * input[r];
        }

        if (i < num_pixels - 1) {
            val += 0.25f * input[r + 3];
        } else {
            val += 0.25f * input[r];
        }

        if (i < num_pixels - 2) {
            val += 0.10f * input[r + 6];
        } else {
            val += 0.10f * input[r];
        }

        if (val > 255.0f) {
            val = 255.0f;
        } else if (val < 0.0f) {
            val = 0.0f;
        }

        output[r] = (uint8_t)val;
        output[r + 1] = input[r + 1];
        output[r + 2] = input[r + 2];
    }
}

// --- Version 2: Planar Kernels ---
// Data layout: Contiguous channels (RRR... GGG... BBB...)
__global__ void invert_planar(uint8_t* green, uint8_t* blue, int num_pixels) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < num_pixels) {
        // Coalesced access: adjacent threads access adjacent memory
        green[i] = 255 - green[i];
        blue[i] = 255 - blue[i];
    }
}

__global__ void gaussian_planar(uint8_t* input, uint8_t* output, int num_pixels) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < num_pixels) {
        float val = 0.50f * input[i];

        if (i >= 1) {
            val += 0.25f * input[i - 1];
        } else {
```

```
            val += 0.25f * input[i];
        }

        if (i >= 2) {
            val += 0.10f * input[i - 2];
        } else {
            val += 0.10f * input[i];
        }

        if (i < num_pixels - 1) {
            val += 0.25f * input[i + 1];
        } else {
            val += 0.25f * input[i];
        }

        if (i < num_pixels - 2) {
            val += 0.10f * input[i + 2];
        } else {
            val += 0.10f * input[i];
        }

        if (val > 255.0f) {
            val = 255.0f;
        } else if (val < 0.0f) {
            val = 0.0f;
        }

        output[i] = (uint8_t)val;
    }
}

int main (void) {
    // Read the image
    uint8_t* image_array = get_image_array();
    uint8_t* new_image_array = (uint8_t*)malloc(M*N*C);

    int num_pixels = M * N;
    uint8_t *d_red, *d_green, *d_blue, *d_red_out;
    cudaMalloc(&d_red, num_pixels);
    cudaMalloc(&d_green, num_pixels);
    cudaMalloc(&d_blue, num_pixels);
    cudaMalloc(&d_red_out, num_pixels);

    // Events for timing
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    // Extract channels on host
    uint8_t *h_red = (uint8_t*)malloc(num_pixels);
    uint8_t *h_green = (uint8_t*)malloc(num_pixels);
    uint8_t *h_blue = (uint8_t*)malloc(num_pixels);
    for(int i=0; i<num_pixels; i++) {
        h_red[i] = image_array[i*3];
        h_green[i] = image_array[i*3+1];
        h_blue[i] = image_array[i*3+2];
    }

    cudaMemcpy(d_red, h_red, num_pixels, cudaMemcpyHostToDevice);
    cudaMemcpy(d_green, h_green, num_pixels, cudaMemcpyHostToDevice);
    cudaMemcpy(d_blue, h_blue, num_pixels, cudaMemcpyHostToDevice);

    // Benchmarking parameters
    int thread_counts[] = {1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024};
    int iterations = 100;

    std::cout << "TPB\tInterleaved_Total\tPlanar_Total (ms)" << std::endl;

    for (int t = 0; t < 11; t++) {
        int tpb = thread_counts[t];
        int bpg = (num_pixels + tpb - 1) / tpb;

        float t_inter=0, t_plan=0;

        for (int i = 0; i < iterations; i++) {
            // Setup device memory for this iteration
            uint8_t *d_inter;
            cudaMalloc(&d_inter, num_pixels * 3);
```

```
            cudaMemcpy(d_inter, image_array, num_pixels * 3, cudaMemcpyHostToDevice);

            // Interleaved Version
            cudaEventRecord(start);
            invert_interleaved<<<bpg, tpb>>>(d_inter, num_pixels);
            gaussian_interleaved<<<bpg, tpb>>>(d_inter, d_inter, num_pixels);
            cudaEventRecord(stop); cudaEventSynchronize(stop);
            float ms;
            cudaEventElapsedTime(&ms, start, stop);
            t_inter += ms;
            cudaFree(d_inter);

            // Planar Version
            cudaMemcpy(d_red, h_red, num_pixels, cudaMemcpyHostToDevice);
            cudaMemcpy(d_green, h_green, num_pixels, cudaMemcpyHostToDevice);
            cudaMemcpy(d_blue, h_blue, num_pixels, cudaMemcpyHostToDevice);

            cudaEventRecord(start);
            invert_planar<<<bpg, tpb>>>(d_green, d_blue, num_pixels);
            gaussian_planar<<<bpg, tpb>>>(d_red, d_red_out, num_pixels);
            cudaEventRecord(stop);
            cudaEventSynchronize(stop);
            cudaEventElapsedTime(&ms, start, stop);
            t_plan += ms;
        }

        std::cout << tpb << "\t"
                  << t_inter/iterations << "\t\t"
                  << t_plan/iterations << std::endl;
    }

    int final_tpb = 256;
    int final_bpg = (num_pixels + final_tpb - 1) / final_tpb;

    // Interleaved Result
    uint8_t *d_inter_final;
    cudaMalloc(&d_inter_final, num_pixels * 3);
    cudaMemcpy(d_inter_final, image_array, num_pixels * 3, cudaMemcpyHostToDevice);
    invert_interleaved<<<final_bpg, final_tpb>>>(d_inter_final, num_pixels);
    gaussian_interleaved<<<final_bpg, final_tpb>>>(d_inter_final, d_inter_final,
        num_pixels);
    cudaMemcpy(new_image_array, d_inter_final, num_pixels * 3, cudaMemcpyDeviceToHost);
    save_image_array(new_image_array, "interleaved.ppm");
    cudaFree(d_inter_final);

    // Planar Result
    cudaMemcpy(d_red, h_red, num_pixels, cudaMemcpyHostToDevice);
    cudaMemcpy(d_green, h_green, num_pixels, cudaMemcpyHostToDevice);
    cudaMemcpy(d_blue, h_blue, num_pixels, cudaMemcpyHostToDevice);

    invert_planar<<<final_bpg, final_tpb>>>(d_green, d_blue, num_pixels);
    gaussian_planar<<<final_bpg, final_tpb>>>(d_red, d_red_out, num_pixels);

    cudaMemcpy(h_red, d_red_out, num_pixels, cudaMemcpyDeviceToHost);
    cudaMemcpy(h_green, d_green, num_pixels, cudaMemcpyDeviceToHost);
    cudaMemcpy(h_blue, d_blue, num_pixels, cudaMemcpyDeviceToHost);

    // Merge results
    for(int i=0; i<num_pixels; i++) {
        new_image_array[i*3] = h_red[i];
        new_image_array[i*3+1] = h_green[i];
        new_image_array[i*3+2] = h_blue[i];
    }
    save_image_array(new_image_array, "planar_result.ppm");

    cudaFree(d_red);
    cudaFree(d_green);
    cudaFree(d_blue);
    cudaFree(d_red_out);
    free(h_red);
    free(h_green);
    free(h_blue);
    free(new_image_array);
    return 0;
}
```

Listing 2: Code voor thread divergence planar vs interleaved

# 3 Results

## 3.1 Warps

| TPB | Total Threads | Avg Time (ms) |
|---|---|---|
| 1 | 230400 | 4.82599 |
| 2 | 230400 | 3.36554 |
| 4 | 230400 | 1.94917 |
| 8 | 230400 | 1.00135 |
| 16 | 230400 | 0.570212 |
| 32 | 230400 | 0.358452 |
| 64 | 230400 | 0.231211 |
| 128 | 230400 | 0.178843 |
| 256 | 230400 | 0.17421 |
| 512 | 230400 | 0.186888 |
| 1024 | 230400 | 0.177462 |

Figure 1: Resultaat voor deel 1 bij verschillende threads

**In gafiek formaat**

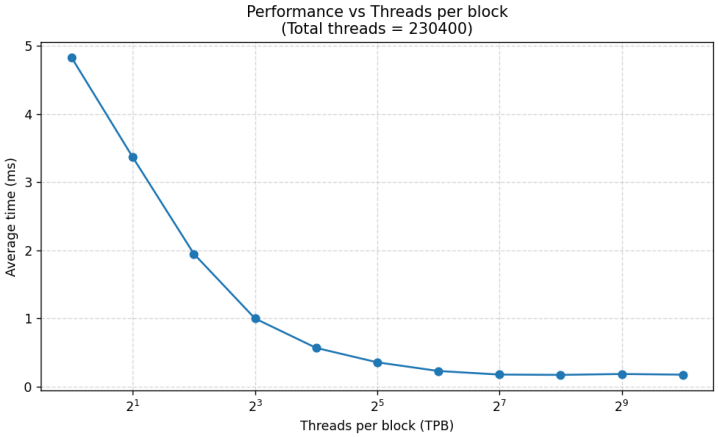



Figure 2: Grafisch voorstelling van resultaat bij warps

## Besluit

Er is een aanzienlijke verbetering is tot 32 threads per blok (TPB). Er is een duidelijke daling te zien tot 16-32 TPB. Pas op dit punt wordt de GPU echt efficiënt omdat de bloks nu volledige warps bevatten.

Er is te zien dat de beste resultaten behaald worden bij 128-256 TPB. De snelste gemiddelde tijd die behaald wordt is 0.174 ms. Bij deze blok grote verkrijgen we de beste combinatie van parallellisme. Bij 512 TPB en hoger wordt het weer wat trager. Omdat dan het aantal cores van de GPU vol zitten.

## 3.2 Divergence

### Resultaat

| TPB | Interleaved_Total | Planar_Total (ms) |
|---|---|---|
| 1 | 14.9641 | 7.32762 |
| 2 | 4.6628 | 3.77706 |
| 4 | 1.90015 | 2.07679 |
| 8 | 1.09049 | 1.19131 |
| 16 | 0.766918 | 0.806045 |
| 32 | 0.543647 | 0.580033 |
| 64 | 0.456132 | 0.413414 |
| 128 | 0.778643 | 0.552588 |
| 256 | 0.377618 | 0.29334 |
| 512 | 0.312539 | 0.276478 |
| 1024 | 0.333777 | 0.294224 |

Figure 3: Resultaat voor deel 2

### Grafiek

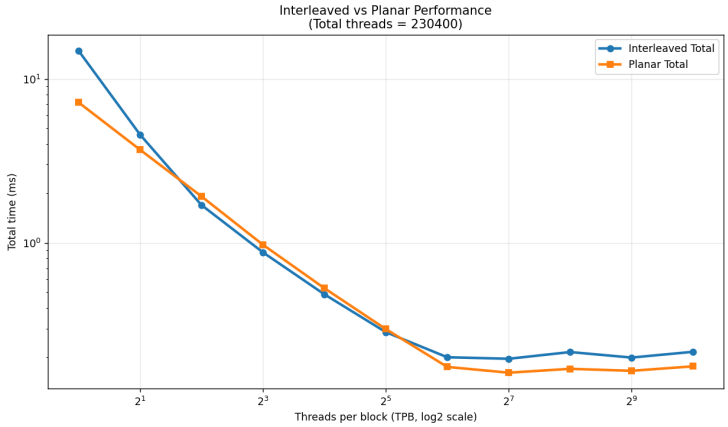



Figure 4: Grafische voorstelling interleaved vs planar

### Staafgrafiek

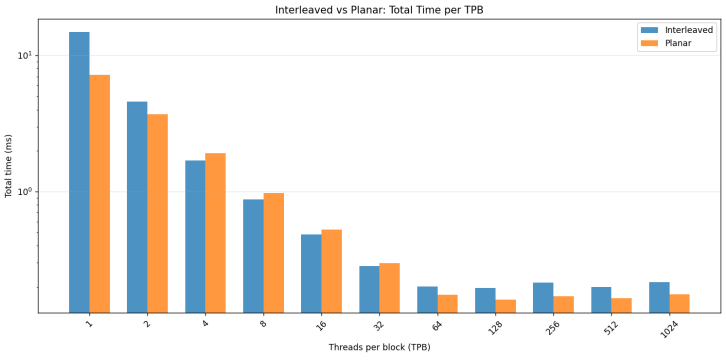



Figure 5: Staafgrafiek voor visuele weergave van tijdverschil

### Besluit

Bij lage TPB (4-32) is het tijdsverschil tussen interleaved en planar klein. Soms is interleaved zelfs iets sneller. Dit is te wijten aan het feit dat de overhead van de kernel launches en synchronisatie domineren. Bij 1 en 2 TPB zien we dat interleaved duidelijk veel sneller is dan planair. De geheugenbrandbreedte zorgt nog niet voor een bottleneck waardoor planar nog niet opweegt tegen over de andere kosten.

Vanaf 64 TPB en hoger is de planaire versie veel sneller. Vanaf dit aantal komen we in het regime waar de GPU echt goed benut wordt. De planaire manier van werken heeft een veel betere geheugentoegang wat zich vertaald in tijdswinst.

Deze tijdsverschillen zijn te zien in zowel de lijngrafiek voorstelling als de staafgrafiek. Er is duidelijk zichtbaar dat planar vooral voor de hogere aantallen in TPB en de lage aantallen een grotere tijdswinst oplevert t.o.v. de interleaved versie.

# 4 Conclusie

In dit labo hebben we twee belangrijke prestatie aspecten van GPU programmeren onderzocht: de invloed van het aantal actieve threads en warps op de uitvoeringstijd en de impact van geheugenlayout (interleaved vs planar).

Uit het eerste deel blijkt dat de inversie kernel aanzienelijk versnelt wanneer het aantal TPB toenemen tot 32-64. Blij lagere TPB waarde 1-16 is het gebruik van de GPU inefficiënt. Omdat er dan onvolledige warps actief zijn. Dit resulteert in overhead en dus tijdsvertraging. De tijdprestaties stabiliseren en bereiken en optimum ronde de 128-256 TPB. Bij hogere TPB 512-1024 neemt de tijd weer toe door overbezetting van het aantal cores.

Dit eerste deel maakte duidelijk wat het belang is van een goede afstemming van het thread blokformaat en dat er rekening dient gehouden te worden met de warp grootte (32 threads).

Het tweede deel toont aan dat de keuze van de data lay-out een grote impact heeft op de prestaties van de GPU. De planaire layout (RRR...GGG...BBB...) zorgt voor betere geheugen samensmelting *(geheugen coalescing)* dit zorgt voor: aangrenzende threads kunnen R/W, aaneengesloten geheugenlocaties. De volle geheugenbrandbreedte wordt maximaal benut hierdoor. Bij interleaved lay-out (RGBRGB...) treedt strided access op. Wat geheugen coalescing sterk vermindert en hierdoor dus ook de geheugenbrandbreedte verlaagt. Uit de simulaties is wordt duidelijk dat: vanaf 64 TPB de planaire implementatie sneller is met een speed-up factor van $\pm 20\%$. Bij lage TPB overheersen de kernel launches, waardoor de tijdswinst klein is of zelfs omkeert.

Dit deel maakt duidelijk dat geheugen coalescing in beeldverwerking vaak grotere winst oplevert dan kleine verschillen in thread divergence.